

PROGRAMACIÓN 1

TRABAJO PRÁCTICO INTEGRADOR

TEMA: GESTOR DE PAÍSES

Alumno: Lucas Rodriguez

Comisión: 26

Profesor: Tomas Garcia

Fecha de entrega: 17/06/2026

---

ÍNDICE

|                                                   |   |
|---------------------------------------------------|---|
| 1. MARCO TEÓRICO .....                            | 2 |
| 1.1 Listas .....                                  | 2 |
| 1.2 Diccionarios .....                            | 2 |
| 1.3 Funciones y modularización .....              | 2 |
| 1.4 Estructuras condicionales y repetitivas ..... | 2 |
| 1.5 Ordenamientos .....                           | 3 |
| 1.6 Estadísticas con funciones built-in .....     | 3 |
| 1.7 Manejo de archivos CSV .....                  | 3 |
| 2. DECISIONES TÉCNICAS .....                      | 3 |
| 2.1 Estructura de datos .....                     | 3 |
| 2.2 Validaciones .....                            | 4 |
| 2.3 Persistencia de datos .....                   | 4 |
| 3. DIFICULTADES Y CONCLUSIONES .....              | 4 |
| 4. BIBLIOGRAFÍA .....                             | 5 |
| 5. LINKS DEL PROYECTO .....                       | 5 |

---

1. MARCO TEÓRICO

1.1 Listas

Una lista en Python es una estructura de datos mutable que permite almacenar una colección ordenada de elementos. En este trabajo se utilizó la lista `países = []` como contenedor principal donde cada elemento es un diccionario que representa un país. Las listas permiten agregar elementos con `append()`, recorrerlos con `for` y acceder por índice.

1.2 Diccionarios

Un diccionario es una estructura que almacena pares clave-valor. Cada país se representa como: `{'nombre': 'Argentina', 'poblacion': 45376763, 'superficie': 2780400, 'continente': 'América'}`. Esto permite acceso directo a los datos por clave y es más

legible que usar índices numéricos. Se utilizaron los métodos `get()` para acceso seguro y `keys()` para validar existencia.

### 1.3 Funciones y modularización

Se aplicó el principio de "una función, una responsabilidad". El código se dividió en funciones como `cargar_paises()`, `agregar_pais()`, `buscar_pais()`, `filtrar_paises()`, etc. Esto facilita el mantenimiento, la lectura y la reutilización de código. Cada función recibe parámetros explícitos y retorna valores cuando corresponde.

### 1.4 Estructuras condicionales y repetitivas

Se utilizaron estructuras `if/elif/else` para validar entradas del usuario y controlar el flujo del menú. El bucle `while True` mantiene el programa en ejecución hasta que el usuario elige salir. Los bucles `for` recorren la lista de países para búsquedas, filtros y estadísticas.

### 1.5 Ordenamientos

Para ordenar se utilizó la función `sorted()` con el parámetro `key=lambda x: x['criterio']`. Esto permite ordenar la lista de diccionarios por cualquier campo. El parámetro `reverse=True` se usa para orden descendente. Ejemplo: `sorted(paises, key=lambda p: p['poblacion'], reverse=True)`.

### 1.6 Estadísticas con funciones built-in

Se utilizaron `max()` y `min()` con parámetro `key` para obtener el país con mayor y menor población sin ordenar toda la lista. Para promedios se usó `sum(p['poblacion'] for p in paises) / len(paises)`. Para contar por continente se implementó un diccionario acumulador.

### 1.7 Manejo de archivos CSV

Se usó el módulo `csv` de la biblioteca estándar. `csv.DictReader` lee el archivo y convierte cada fila en un diccionario automáticamente. `csv.DictWriter` escribe la lista de diccionarios al archivo usando `writeheader()` y `writerows()`. Se implementó manejo de errores con `try/except` para evitar que el programa se detenga si el CSV está corrupto.

## 2. DECISIONES TÉCNICAS

### 2.1 Estructura de datos

Se eligió una lista de diccionarios porque combina la flexibilidad de las listas para almacenar múltiples registros con la claridad de los diccionarios para acceder a campos por nombre. Alternativas como lista de listas requerirían recordar posiciones de columnas y serían más propensas a errores.

### 2.2 Validaciones

Todas las entradas numéricas se validan con `try/except` para capturar `ValueError` si el usuario ingresa texto. Los campos de texto se validan con `strip()` para eliminar espacios y se verifica que no estén vacíos. Para evitar duplicados se usa `any(p['nombre'].lower() == nuevo.lower() for p in paises)` antes de agregar.

### 2.3 Persistencia de datos

Se optó por sobrescribir el archivo CSV completo en cada operación de alta, baja o modificación mediante `guardar_paises()`. Si bien no es óptimo para grandes volúmenes de datos, para este TP con menos de 100 registros es simple y efectivo. Se garantiza que no se pierde información si el programa se cierra abruptamente.

### 3. DIFICULTADES Y CONCLUSIONES

La principal dificultad encontrada fue el manejo de errores al leer el archivo CSV. Inicialmente el programa se detenía si una línea tenía formato incorrecto. Se resolvió implementando try/except dentro del bucle de lectura, permitiendo omitir registros corruptos y continuar con los válidos, informando al usuario cuántos se cargaron correctamente.

Otra dificultad fue implementar la actualización parcial de un país. Se resolvió usando el operador or para mantener el valor anterior si el usuario deja el campo vacío: `poblacion = int(nueva_pob) if nueva_pob else pais['poblacion']`.

Como conclusión, este trabajo permitió afianzar el uso combinado de listas y diccionarios, que es fundamental en Python. Se comprendió la importancia de la modularización para mantener código legible y la validación exhaustiva de entradas para crear programas robustos. El uso de archivos CSV demostró ser una forma simple de lograr persistencia sin bases de datos.

### 4. BIBLIOGRAFÍA

- Python Software Foundation. (2026). Documentación oficial de Python 3. Recuperado de <https://docs.python.org/3/>
- Módulo CSV. Documentación oficial. Recuperado de <https://docs.python.org/3/library/csv.html>
- Material de cátedra Programación 1. UTN TUP. Campus Virtual 2026.

### 5. LINKS DEL PROYECTO

Repositorio GitHub: <https://github.com/lucas-rodriquez08/UTN-TPI-PROGRAMACION-LUCAS-RODRIGUEZ>

Video demostración: <https://youtu.be/OQzxApY7cxk>